BUILD SUMMARY

Client & Admin Frontends, Real-Time Messenger, and Lawyer AI Assist

A single working session delivering three product surfaces, a WebSocket real-time messaging system across two apps, and an AI assist layer for the lawyer-side conversation — all built on the platform's existing legal-AI services.

3	2	4	1
Product surfaces	Apps made live	AI features	WebSocket layer

Scope: client portal & admin frontends brought up, a client registration bug fixed permanently, lawyer-side case creation, a full messenger overhaul (live updates, media, paste, drag-drop, lightbox), and AI assistance for the lawyer.

1 · Client & Admin Frontends

Both the client portal and the admin dashboard were brought up and made viewable, and a blocking client-registration failure was diagnosed and fixed at the root rather than patched over.

Client Portal live

Vite dev server running; portal UI verified end-to-end.

Admin Dashboard live

Separate app served and verified on its own port.

Registration 404 fixed

Root cause: tenant slug data + strict lookup. Added a default tenant fallback and backfilled all NULL slugs — permanent fix.

Lawyer case creation

"New Case" modal wired to the existing tenant-scoped API; lawyer is the correct owner of case creation.

2 · Real-Time Messenger (both apps)

The messenger went from "reload to see new messages" to a true real-time experience on both the lawyer app and the client portal, with rich media handling.

- **WebSocket real-time delivery** — new backend WS layer (per-case rooms, JWT-authed) with auto-reconnect; polling kept only as a fallback when the socket is down.
- **Optimistic send** — messages appear instantly with sending / failed / retry states.
- **Media like a chat app** — inline images & video, paste screenshots, drag-and-drop, multi-file queue with captions.
- **Gallery lightbox** — blurred backdrop, arrow-key navigation across all thread images, download, Esc to close.
- **Smart scroll & unread divider** — no scroll-jump on updates; “new messages” pill and a last-read divider.
- **Typing indicator** — live over WebSocket.
- **Bug fixed along the way** — portal attachment downloads were unauthenticated and broken; now fetched with auth.

3 · Lawyer AI Assist

An AI layer added to the messenger, lawyer-facing first, reusing the platform’s existing legal-AI services (LLM gateway, PII redactor, case context, document insight). Nothing is auto-sent.

■ Suggest replies

3 grounded reply options from thread + case context. Lawyer clicks one into the composer and edits before sending.

■ Summarize thread

Long conversations collapsed into bullet points for fast context recovery.

■ PII guard (both sides)

Pre-send scan flags emails / phones / IDs with a review-or-send-anyway prompt. Also added to the client portal.

■ Document insight

One-click analysis of files a client shares: type, summary, parties, key points — powered by existing DocumentInsight.

Guardrails built in. Every AI call fails open with a safe fallback (never a broken UI or blocked send). Suggestions are drafts only — the lawyer always reviews and edits. The model is prompted not to give legal conclusions, and a visible disclaimer is shown. Existing trust / redaction services were reused rather than duplicated.

Quality & Verification

- Lawyer app: TypeScript build clean (tsc).
- Client portal: TypeScript build clean (tsc).
- Backend: syntax-validated across all modified modules.
- Existing services reused; no parallel logic introduced.

Known Caveats & Next Steps

- WebSocket layer is in-process (single worker). Multi-worker deployments need a Redis pub/sub backplane; polling fallback keeps it functional meanwhile.
- AI suggest/summarize use the standard LLM tier directly, not the full copilot RAG pipeline — fast, but less deeply grounded than the case Assistant.
- Attachment insight is richest when the chat file is also persisted as a case Document; wiring chat uploads into the document pipeline is a strong follow-up.
- Read receipts still update on poll/open, not pushed live.

Generated for the Legal AI Platform — internal build record. All work verified at session close.